# DATA SCIENCE LAB EXAM STUDY GUIDE

Comprehensive, Beginner-Friendly, Exam-Ready Theory, Algorithms, Code Breakdowns & Viva Reference

**Repository:** `ds` | **Target:** Practical Examination Writing & Execution

---

## Table of Contents


1. Master Coding Flow & Core Paradigms — Section 1
2. General Algorithms for 7 Core Coding Patterns — Section 2
3. Master Program Comparison Table (All 10 Programs) — Section 3
4. Program 01: Introduction to NumPy (e01_introduction_to_numpy.py) — Section 4.1
5. Program 02: Matrix Operations (e02_matrix_operations.py) — Section 4.2
6. Program 03: Matplotlib Visualizations (e03_matplotlib.py) — Section 4.3
7. Program 04: Pandas DataFrames & Series (e04_pandas.py) — Section 4.4
8. Program 05: K-Nearest Neighbors Classifier (e05_knn.py) — Section 4.5
9. Program 06: Naive Bayes Classification (e06_naive_bayes.py) — Section 4.6
10. Program 07: Decision Tree Classifier (e07_decision_tree.py) — Section 4.7
11. Program 08: Naive Bayes with User Input (e08_naive_bayes.py) — Section 4.8
12. Program 09: K-Means Clustering (e09_kmeans.py) — Section 4.9
13. Program 10: Multiple Linear Regression (e10_multiple_linear_regression.py) — Section 4.10
14. Comparative Analysis (KNN vs NB vs Decision Tree vs Regression vs KMeans) — Section 5
15. Final Pre-Exam Checklist & Rapid Revision Sheet — Section 6
16. Review Notes & Observations on Repository Files — Section 7

## Section 1: Master Coding Flow & Core Paradigms

In college Data Science lab exams, programs fall into distinct categories. Recognizing which category an exam question belongs to allows you to instantly recall its structure and expected outputs.

### The Universal 5-Step Machine Learning Pipeline

Programs **e05, e06, e07, e08, and e10** strictly adhere to the universal machine-learning pipeline:

```
[1. Import Libraries]  -->  import numpy, pandas, matplotlib, sklearn
[2. Load Dataset]      -->  load_iris(), load_diabetes(), or read_csv()
[3. Prepare Data]      -->  Separate features (X) and target (y); train_test_split()
[4. Model Pipeline]    -->  Initialize model --> .fit(X_train, y_train) --> .predict(X_test)
[5. Evaluation/Plot]   -->  accuracy_score(), classification_report, MSE, R2, or plt.show()
```

### Core Task Paradigms Compared

| Paradigm | Type | Target Variable (y)? | Core Goal | Repository Programs | Key Evaluation Metrics |
|---|---|---|---|---|---|
| **Supervised Classification** | Supervised | Yes (Discrete classes: 0, 1, 2) | Assign inputs to categorical labels (e.g. Setosa, Versicolor, Virginica) | `e05_knn.py`<br>`e06_naive_bayes.py`<br>`e07_decision_tree.py`<br>`e08_naive_bayes.py` | Accuracy Score, Confusion Matrix, Classification Report (Precision, Recall, F1) |
| **Supervised Regression** | Supervised | Yes (Continuous numbers) | Predict a continuous numerical value along a best-fit hyperplane | `e10_multiple_linear_regression.py` | Mean Squared Error (MSE), Coefficient of Determination ($R^2$ Score) |
| **Unsupervised Clustering** | Unsupervised | **No** (No ground truth labels) | Discover natural groupings based on spatial proximity / distances | `e09_kmeans.py` | Inertia (Within-cluster sum of squares), Centroid positions, Cluster Labels |
| **Data Wrangling & Math** | Foundational | N/A | Array transformations, linear algebra, tabular manipulation, charts | `e01`, `e02`, `e03`, `e04` | Correct mathematical output, clean visual rendering |

## Section 2: General Algorithms for 7 Core Coding Patterns

Before studying individual scripts, memorize the 7 fundamental algorithms below. In an exam, if you are asked to write an algorithm for any category, these provide the exact, examiner-approved step-by-step structure.

### Pattern 1: General Algorithm for a NumPy Program

**Applies to:** Array initialization, mathematical transformations, indexing, boolean masking, and file I/O (e.g., `e01_introduction_to_numpy.py`).

**ALGORITHM (Exam Writing Format):**

1. **START**
2. Import the NumPy library with the standard alias `np`.
3. Create or initialize NumPy arrays from Python lists using `np.array()`, or generate numbers using functions like `np.arange()`, `np.random.rand()`, or `np.random.randint()`.
4. Perform element-wise operations, conversions (e.g. `np.deg2rad()`), or calculations using universal functions (e.g. `np.diff()`, `np.mean()`, `np.var()`, `np.std()`).
5. Filter or search array elements using boolean indexing (e.g. `arr[arr > value]`) or search conditions using `np.where()`.
6. Perform axis-wise reductions if processing 2D matrices using `np.sum(axis=0)` (columns) or `np.sum(axis=1)` (rows).
7. Optionally save the array to a text file using `np.savetxt()` and reload it using `np.loadtxt()`.
8. Display the resulting arrays and computed statistical metrics to the console.
9. **STOP**

**Repository Syntax Example:**

```
import numpy as np
arr = np.random.randint(low=-40, high=40, size=10)
filtered = arr[arr > 4]
np.savetxt("test.txt", arr)
loaded = np.loadtxt("test.txt")
```

### Pattern 2: General Algorithm for a Matrix Operations Program

**Applies to:** User-input matrices, determinant evaluation, rank, trace, and inversion (e.g., `e02_matrix_operations.py`).

**ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `numpy` and the linear algebra sub-package `linalg`.
3. Define a helper function to read an $n imes n$ square matrix by taking dimensions and reading elements row-by-row through nested loops.
4. Prompt the user to enter the matrix order $n$ and invoke the helper function to construct matrices `matrix1` and `matrix2` as NumPy arrays.
5. Compute the matrix dot product using `np.dot()` and matrix transpose using `np.transpose()`.
6. Calculate the matrix trace (sum of main diagonal elements) using `np.trace()`.
7. Compute the matrix rank using `linalg.matrix_rank()`.
8. Evaluate the matrix determinant using `np.linalg.det()`.
9. Check whether the determinant is non-zero; if non-zero, compute the inverse matrix using `linalg.inv()`, otherwise report that the inverse does not exist.
10. Compute the eigenvalues and corresponding eigenvectors using `linalg.eig()`.
11. Display all computed linear algebra results cleanly to the user.
12. **STOP**

**Repository Syntax Example:**

```
import numpy as np
from numpy import linalg
dot_p = np.dot(matrix1, matrix2)
dt = np.linalg.det(matrix1)
if dt != 0: inv_m = linalg.inv(matrix1)
vals, vecs = linalg.eig(matrix1)
```

### Pattern 3: General Algorithm for a Matplotlib Program

**Applies to:** Line plots, multi-line charts, bar charts, pie charts, scatter plots, and subplots (e.g., `e03_matplotlib.py`).

**ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `matplotlib.pyplot` as `plt`, along with `numpy` and `pandas` if required for data handling.
3. Prepare or load coordinates and values from static arrays, user collections, or external text/CSV files (e.g., using `open()` or `pd.read_csv()`).
4. Configure the figure dimensions or subplot grid if drawing multiple plots using `plt.figure(figsize=(w, h))` and `plt.subplot(nrows, ncols, index)`.
5. Invoke the target plotting function:
   - Line plot: `plt.plot(x, y, color=..., linestyle=..., marker=...)`
   - Bar charts: `plt.bar(categories, values)` or horizontal `plt.barh()`
   - Pie chart: `plt.pie(values, labels=..., autopct="%1.1f%%")`
   - Scatter plot: `plt.scatter(x, y, color=..., marker=...)`
6. Add descriptive visual titles and annotations using `plt.title()`, `plt.xlabel()`, and `plt.ylabel()`.
7. If multiple series or groups are plotted, call `plt.legend()` to display the labeled key.
8. Call `plt.show()` to display the window and render the figure.
9. **STOP**

**Repository Syntax Example:**

```
import matplotlib.pyplot as plt
plt.plot(x, y, color="red", linestyle=":", marker=".", label="Highs")
plt.xlabel("Day"); plt.ylabel("Temp"); plt.title("Forecast"); plt.legend()
plt.show()
```

### Pattern 4: General Algorithm for a Pandas Tabular Data Program

**Applies to:** Series conversions, date ranges, sorting, index resetting, grouping, aggregation, missing-value imputation, and DataFrame merging (e.g., `e04_pandas.py`).

**ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `pandas` as `pd` and `numpy` as `np`.
3. Create Series or DataFrames from Python lists, dictionaries, or read external CSV files using `pd.read_csv("filename.csv")`.
4. Perform date sequence generation using `pd.date_range()` with start and end dates.
5. Sort data across single or multiple columns with `df.sort_values(by=[...], ascending=[...])`.
6. Slice rows and columns using index-based locations via `df.iloc[row_start:row_end, col_start:col_end]` or reset custom indices using `df.reset_index(drop=True)`.
7. Group data by categorical columns and apply aggregate functions using `df.groupby("Category")["TargetCol"].mean()`.
8. Identify missing data and impute missing values using `df.fillna(value)`.
9. Merge two separate DataFrames on a common key column using `pd.merge(df1, df2, on="ID", how="inner")`.
10. Display the resulting tabular structures to the console using `print()`.
11. **STOP**

**Repository Syntax Example:**

```
import pandas as pd
df = pd.read_csv("medal.csv")
sorted_df = df.sort_values(by=["Score", "Name"], ascending=[False, True])
avg_by_job = df.groupby("Job")["Salary"].mean()
merged = pd.merge(df1, df2, how="inner", on="ID")
```

### Pattern 5: General Algorithm for Supervised Classification

**Applies to:** KNN, Naive Bayes, Decision Trees (e.g., `e05_knn.py`, `e06_naive_bayes.py`, `e07_decision_tree.py`, `e08_naive_bayes.py`).

**ALGORITHM (Exam Writing Format):**

1. **START**
2. Import required modules from `sklearn`: dataset loaders (`load_iris`), model selection (`train_test_split`), model classifier classes (`KNeighborsClassifier`, `GaussianNB`, or `DecisionTreeClassifier`), and evaluation `metrics`.
3. Load the dataset into memory and extract the feature matrix $X$ (measurements) and target vector $y$ (class labels).
4. Partition the data into training and testing subsets using `train_test_split(x, y, test_size=..., random_state=...)`.
5. Instantiate the chosen classifier with required parameters (e.g., `n_neighbors=5` or `criterion="entropy"`).
6. Train the model on the training set using `model.fit(x_train, y_train)`.
7. Generate predictions on the unseen test set using `y_pred = model.predict(x_test)`.
8. Evaluate classification accuracy using `metrics.accuracy_score(y_test, y_pred)` and generate precision/recall statistics using `metrics.classification_report()`.
9. Optionally predict the class of a new test sample or user input, and visualize predictions or model structure using Matplotlib.
10. **STOP**

**Repository Syntax Example:**

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics

iris = load_iris(); x, y = iris.data, iris.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=40)
gnb = GaussianNB().fit(x_train, y_train)
y_pred = gnb.predict(x_test)
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

### Pattern 6: General Algorithm for Unsupervised K-Means Clustering

**Applies to:** Partitioning unlabeled continuous data into $k$ distinct spatial clusters (e.g., `e09_kmeans.py`).

**ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `numpy`, `KMeans` from `sklearn.cluster`, synthetic generator `make_blobs` from `sklearn.datasets`, and `matplotlib.pyplot` as `plt`.
3. Generate or load unlabelled continuous coordinate data $X$ without true target labels $y$ (e.g. using `make_blobs(n_samples=300)`).
4. Instantiate the `KMeans` estimator specifying the target number of clusters $k$ (e.g., `n_clusters=5`, `random_state=42`, `n_init="auto"`).
5. Fit the K-Means model directly to the feature data using `kmeans.fit(data)`.
6. Retrieve the discovered cluster assignments for each sample via `kmeans.labels_`.
7. Retrieve the final spatial centroid coordinates for each cluster via `kmeans.cluster_centers_`.
8. Render a 2D scatter plot of the data points, mapping point colors to cluster labels (`c=labels, cmap="viridis"`).
9. Overlay the cluster centroids onto the scatter plot as distinct markers (e.g. red `'x'` markers).
10. Predict cluster membership for arbitrary new coordinate points using `kmeans.predict([[x1, y1], [x2, y2]])`.
11. Display the visual cluster plot using `plt.show()` and print predicted clusters to the console.
12. **STOP**

**Repository Syntax Example:**

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
data, _ = make_blobs(n_samples=300)
kmeans = KMeans(n_clusters=5, random_state=42, n_init="auto").fit(data)
labels = kmeans.labels_
centroids = kmeans.cluster_centers_
pred_clusters = kmeans.predict([[2, -4], [6, 6], [6, -2]])
```

### Pattern 7: General Algorithm for Multiple Linear Regression

**Applies to:** Continuous value modeling using multiple independent variables (e.g., `e10_multiple_linear_regression.py`).

**ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `numpy`, `matplotlib.pyplot`, `datasets` and `linear_model` from `sklearn`, and evaluation metrics `mean_squared_error` and `r2_score` from `sklearn.metrics`.
3. Load the continuous regression dataset using `datasets.load_diabetes(return_X_y=True)` to unpack feature matrix $X$ and target array $y$.
4. Select multiple specific predictor feature columns (e.g. column 0 for Age and column 2 for BMI) using NumPy slicing: `diabetes_X[:, [0, 2]]`.
5. Partition the dataset into training and testing sets by reserving fixed slices (e.g., all rows except last 20 for training: `[:-20]`, last 20 for testing: `[-20:]`).
6. Instantiate the linear regression model: `regr = linear_model.LinearRegression()`.
7. Fit the regression model using the training inputs and target progression values: `regr.fit(diabetes_X_train, diabetes_y_train)`.
8. Generate continuous target predictions on the test set: `diabetes_y_pred = regr.predict(diabetes_X_test)`.
9. Extract and print the mathematical model parameters: slopes/weights via `regr.coef_` and the y-intercept bias via `regr.intercept_`.
10. Evaluate model accuracy using `mean_squared_error(y_test, y_pred)` and $R^2$ determination score using `r2_score(y_test, y_pred)`.
11. Plot actual vs predicted responses across individual features and display the ideal fit line ($y=x$) using Matplotlib.
12. **STOP**

**Repository Syntax Example:**

```
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score
diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)
diabetes_X = diabetes_X[:, [0, 2]]
regr = linear_model.LinearRegression().fit(diabetes_X[:-20], diabetes_y[:-20])
pred = regr.predict(diabetes_X[-20:])
print("MSE:", mean_squared_error(diabetes_y[-20:], pred))
print("R2:", r2_score(diabetes_y[-20:], pred))
```

## Section 3: Master Program Comparison Table

Quick reference overview of all 10 Python files in the repository:

| # | Filename | Primary Library | Core Purpose & Task | Data Source | Key Function / Model | Primary Output | Learning Type |
|---|---|---|---|---|---|---|---|
| **e01** | `e01_introduction_to_numpy.py` | NumPy | Array stats, slicing, filtering, file I/O | Code arrays, random generator, `test.txt` | `np.mean`, `np.diff`, `np.where`, `np.savetxt` | Console statistical prints | Foundatio Math |
| **e02** | `e02_matrix_operations.py` | NumPy (linalg) | 7 Linear algebra matrix operations | Interactive user console input | `np.dot`, `transpose`, `det`, `inv`, `eig` | Computed matrix metrics | Linear Algebra |
| **e03** | `e03_matplotlib.py` | Matplotlib | 10 standard charting exercises | Static lists, `exercise3.3.txt`, `medal.csv` | `plt.plot`, `plt.bar`, `plt.pie`, `plt.scatter` | 10 Graphical plot windows | Data Visualizati |
| **e04** | `e04_pandas.py` | Pandas | 12 tabular DataFrame transformations | In-memory dictionaries, lists, `medal.csv` | `sort_values`, `iloc`, `groupby`, `fillna`, `merge` | Printed DataFrames & Series | Data Wrangling |
| **e05** | `e05_knn.py` | Scikit-Learn | KNN classification with side-by-side plots | Iris Dataset (4 features, 3 classes) | `KNeighborsClassifier(n_neighbors=5)` | Accuracy + 2 Scatter Subplots | Supervisec Classificati |
| **e06** | `e06_naive_bayes.py` | Scikit-Learn | Gaussian Naive Bayes classification | Iris Dataset | `GaussianNB()` | Accuracy + Classification Report | Supervisec Classificati |
| **e07** | `e07_decision_tree.py` | Scikit-Learn | Decision tree classification & tree diagram | Iris Dataset | `DecisionTreeClassifier(criterion="entropy")` | Classification Report + Tree Plot | Supervisec Classificati |
| **e08** | `e08_naive_bayes.py` | Scikit-Learn | Naive Bayes with interactive user input | Iris Dataset + Live User Floats | `naive_bayes.GaussianNB()` | Accuracy % + Predicted Flower | Supervisec Classificati |
| **e09** | `e09_kmeans.py` | Scikit-Learn | K-Means clustering on 2D synthetic blobs | `make_blobs(n_samples=300)` | `KMeans(n_clusters=5, n_init="auto")` | Cluster scatter plot + Predicted IDs | Unsupervis Clustering |
| **e10** | `e10_multiple_linear_regression.py` | Scikit-Learn | Multiple linear regression on Age & BMI | Diabetes Dataset (Continuous progression) | `linear_model.LinearRegression()` | Coefs, Intercept, MSE, R2, Plots | Supervisec Regressior |

## Section 4: Comprehensive Guide for All 10 Programs

### Program 01: `e01_introduction_to_numpy.py`

**Program 1** **Aim:** To write and execute a NumPy program demonstrating array creation, temperature conversion, statistical evaluations, array sorting, searching, file operations, and axis reductions.

**GENERAL ALGORITHM (Exam Writing Format):**

1. **START**
2. Import the NumPy library as `np`.
3. Define an array of Fahrenheit temperatures `fht`, iterate through each temperature, compute Centigrade using formula $( ext{temp} - 32) imes (5/9)$, append to `degree` array, and print.
4. Generate an array of 1000 random uniform values using `np.random.rand(1000)` and compute its mean, variance, and standard deviation using `np.mean()`, `np.var()`, and `np.std()`.
5. Create an array of angle degrees and convert them to radians using `np.deg2rad()`.
6. Compute element-wise differences between neighbouring elements of an array using `np.diff()`.
7. Define an array of complex numbers and sort them lexicographically (real part first, then imaginary) using `np.sort()`.
8. Generate a 10-element random integer array in range $[-40, 40)$ and extract elements greater than 4 using boolean masking `arr[arr > 4]`.
9. Generate a 10-element random integer array in range $[-10, 40)$, prompt the user for an integer number, and search its presence using `np.where(arr == num)`.
10. Save an array of floats to a text file `test.txt` using `np.savetxt()`, reload it using `np.loadtxt()`, and print the loaded array.
11. Create an array of even integers from 30 to 60 using `np.arange(30, 61, 2)`.
12. Create a $3 imes 3$ integer matrix, compute the grand sum using `np.sum(arr)`, column-wise sums using `np.sum(arr, axis=0)`, and row-wise sums using `np.sum(arr, axis=1)`.
13. **STOP**

#### Code Explained in Logical Blocks

```
# Block 1: Fahrenheit to Centigrade Conversion (Q1)
import numpy as np
fht = np.array([100, 98.4, 91.50, 99.78, 101.5])
degree = np.array([])
for temp in fht:
    degreeval = (temp - 32) * (5 / 9)
    degree = np.append(degree, degreeval)
print("Fahrenheit values", fht)
print("Centigrade values", degree)

# Block 2: Random Statistics & Trigonometry (Q2, Q3, Q4)
arr = np.random.rand(1000)
print("Average =", np.mean(arr))
print("Variance =", np.var(arr))
print("Standard deviation =", np.std(arr))
degrees = np.array([32, 40, 180, 360, 45, 93])
print("Radian values =", np.deg2rad(degrees))
arr = np.array([12, 34, 6, 28, 46, 90, 3])
print("Difference =", np.diff(arr))

# Block 3: Complex Sort, Filtering, and Search (Q5, Q6, Q7)
arr_complex = np.array([complex(7, 5), complex(4, 3), complex(12, 2), complex(1, 8), complex(1, 9)])
print("After sorting =", np.sort(arr_complex))
arr_rand = np.random.randint(low=-40, high=40, size=10)
print("Elements greater than 4 =", arr_rand[arr_rand > 4])
arr_search = np.random.randint(low=-10, high=40, size=10)
num = int(input("Enter a number: "))
res = np.where(arr_search == num)
print("Number is present" if len(res[0]) > 0 else "Number is not present")

# Block 4: File I/O, Sequences, and Reductions (Q8, Q9, Q10)
arr_file = np.array([4., 5., 4., 6., 11., 2235., 56.])
np.savetxt("test.txt", arr_file)
print("Loaded array =", np.loadtxt("test.txt"))
print("Even integers 30-60:", np.arange(30, 61, 2))
mat = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Sum of all elements =", np.sum(mat))
print("Sum of each column =", np.sum(mat, axis=0))
print("Sum of each row =", np.sum(mat, axis=1))
```

#### Important Syntax to Memorize

- `np.deg2rad(degrees)` — Converts degree array directly to radian floating values.
- `np.diff(arr)` — Computes adjacent differences: $arr[i+1] - arr[i]$.
- `np.where(arr == num)` — Returns a tuple of index arrays matching the search condition.
- `np.savetxt("test.txt", arr)` & `np.loadtxt("test.txt")` — Text file persistence.
- `np.sum(arr, axis=0)` (columns) & `np.sum(arr, axis=1)` (rows).

### Expected Output Format

```
Fahrenheit values [100.   98.4  91.5  99.78 101.5 ]
Centigrade values [37.77777778 36.88888889 33.05555556 37.65555556 38.61111111]
Average = 0.501... (Random float around 0.5)
Variance = 0.083... (Random float around 0.08)
Standard deviation = 0.288... (Random float around 0.28)
Degrees = [ 32  40 180 360  45  93]
Radian values = [0.55850536 0.6981317  3.14159265 6.28318531 0.78539816 1.6231562 ]
Difference = [ 22 -28  22  18  44 -87]
Elements greater than 4 = [ ... random integers > 4 ... ]
Enter a number: 15
Number is present in the array (or not present)
Loaded array = [4.000e+00 5.000e+00 4.000e+00 6.000e+00 1.100e+01 2.235e+03 5.600e+01]
Array of all the even integers from 30 to 60:
[30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60]
Sum of all elements = 45
Sum of each column = [12 15 18]
Sum of each row = [ 6 15 24]
```

**Memory Trick: "F-R-D-D-S: Fahrenheit, Random stats, Degrees to radians, Differences, Save/load"**

### Common Lab-Exam Mistakes

- Confusing axes: Writing `axis=1` for columns instead of `axis=0`.
- Writing `np.arange(30, 60, 2)` which stops at 58. It must be `np.arange(30, 61, 2)` to include 60.
- Checking `if res:` instead of checking `if len(res[0]) == 0:` for `np.where`.

**Viva Q1: What is the difference between np.random.rand() and np.random.randint()?**

**Answer:** `np.random.rand()` generates uniform floating-point numbers between 0 and 1, whereas `np.random.randint(low, high, size)` generates discrete integers in the range $[low, high)$.

**Viva Q2: How does np.sort() handle complex numbers?**

**Answer:** NumPy sorts complex numbers lexicographically: it sorts primarily by the real part; if real parts are equal, it sorts by the imaginary part.

## Program 02: `e02_matrix_operations.py`

**Program 2** **Aim:** To write a NumPy program that reads two matrices from the user and computes their dot product, transpose, trace, rank, determinant, inverse, and eigenvalues/eigenvectors.

**GENERAL ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `numpy` as `np` and import `linalg` from `numpy`.
3. Define a function `matrixread(n)` that initializes an empty list `matrix`, loops $i$ from 0 to $n-1$, loops $j$ from 0 to $n-1$, prompts for element $i,j$, appends to a `row`, and appends `row` to `matrix`.
4. Prompt the user to enter the order of square matrix $n$ as an integer.
5. Call `matrixread(n)` twice to read elements for `matrix1` and `matrix2`, converting them into NumPy arrays.
6. Print `matrix1` and `matrix2`.
7. Compute and display the Dot Product using `np.dot(matrix1, matrix2)`.
8. Compute and display the Transpose of `matrix1` using `np.transpose(matrix1)`.
9. Compute and display the Trace (sum of diagonal elements) of `matrix1` using `np.trace(matrix1)`.
10. Compute and display the Rank of `matrix1` using `linalg.matrix_rank(matrix1)`.
11. Compute and display the Determinant of `matrix1` using `np.linalg.det(matrix1)`.
12. Check if `np.linalg.det(matrix1) != 0`: if true, compute and display inverse using `linalg.inv(matrix1)`; otherwise print "Matrix 1 does not have an inverse."
13. Compute eigenvalues and eigenvectors using `linalg.eig(matrix1)` and print both.
14. **STOP**

### Code Explained in Logical Blocks

```
# Block 1: Imports and Matrix Input Helper Function
import numpy as np
from numpy import linalg

def matrixread(n):
    matrix = []
    for i in range(n):
        row = []
        for j in range(n):
            num = int(input("Enter element " + str(i) + "," + str(j) + ": "))
            row.append(num)
        matrix.append(row)
    return matrix

# Block 2: Reading Order and Creating Matrices
n = int(input("Enter the order of the matrix: "))
print("Enter Matrix 1")
matrix1 = np.array(matrixread(n))
print("Enter Matrix 2")
matrix2 = np.array(matrixread(n))
print("Matrix 1:
", matrix1)
print("Matrix 2:
", matrix2)

# Block 3: Computing 7 Linear Algebra Properties
print("Dot Product:
", np.dot(matrix1, matrix2))
print("Transpose of Matrix 1:
", np.transpose(matrix1))
print("Trace of Matrix 1:
", np.trace(matrix1))
print("Rank of Matrix 1:
", linalg.matrix_rank(matrix1))
print("Determinant of Matrix 1:
", np.linalg.det(matrix1))

if np.linalg.det(matrix1) != 0:
    print("Inverse of Matrix 1:
", linalg.inv(matrix1))
else:
    print("Matrix 1 does not have an inverse.")

eigenvalues, eigenvectors = linalg.eig(matrix1)
print("Eigen values:
", eigenvalues)
print("Eigen vectors:
", eigenvectors)
```

### Important Syntax to Memorize

- `np.dot(matrix1, matrix2)` — Matrix multiplication.
- `np.trace(matrix1)` — Sum of elements on the main diagonal.
- `linalg.matrix_rank(matrix1)` — Number of linearly independent rows/columns.
- `np.linalg.det(matrix1)` — Determinant scalar value.
- `linalg.inv(matrix1)` — Matrix inverse ($A^{-1}$ such that $A \cdot A^{-1} = I$).
- `eigenvalues, eigenvectors = linalg.eig(matrix1)` — Unpacks values and vectors.

**Expected Output Format (For $2 imes 2$ matrices with elements 1, 2, 3, 4 and 5, 6, 7, 8)**

```
Matrix 1:
[[1 2]
 [3 4]]
Matrix 2:
[[5 6]
 [7 8]]
Dot Product:
[[19 22]
 [43 50]]
Transpose of Matrix 1:
[[1 3]
 [2 4]]
Trace of Matrix 1:
5
Rank of Matrix 1:
2
Determinant of Matrix 1:
-2.0000000000000004
Inverse of Matrix 1:
[[-2.   1. ]
 [ 1.5 -0.5]]
Eigen values:
[-0.37228132  5.37228132]
Eigen vectors:
[[-0.82456484 -0.41597356]
 [ 0.56576746 -0.90937671]]
```

> **Memory Trick: "D-T-T-R-D-I-E: Dot, Transpose, Trace, Rank, Determinant, Inverse, Eigenvalues"**

### Common Lab-Exam Mistakes

- Trying to unpack `linalg.eig()` into a single variable instead of two (`values, vectors`).
- Forgetting to convert the Python list returned by `matrixread()` into a NumPy array using `np.array()`.

**Viva Q1: What is the trace of a matrix?**

**Answer:** The trace is the sum of the elements along the main diagonal of a square matrix.

**Viva Q2: When is a matrix non-invertible?**

**Answer:** A matrix is non-invertible (singular) when its determinant is zero ($\det(A) = 0$).

## Program 03: `e03_matplotlib.py`

**Program 3** **Aim:** To write programs using Matplotlib to draw line charts, multiple lines with legends, file-based plots, subplots, bar charts, pie charts, and scatter plots.

**GENERAL ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `matplotlib.pyplot` as `plt`, `numpy` as `np`, and `pandas` as `pd`.
3. **Q1 Line Plot:** Define coordinate arrays $x=[1, 2, 6, 18]$, $y=[3, 10, 12, 20]$, plot red dotted line with green dots using `plt.plot()`, and call `plt.show()`.
4. **Q2 Labeled Line:** Plot Temperature vs Sales using green line with `marker="*"`, apply `plt.xlabel()` and `plt.ylabel()`, and call `plt.show()`.
5. **Q3 File Plot:** Open `exercise3.3.txt`, read two comma-separated lines for $x$ and $y$, convert to integer NumPy arrays, close file, plot with title and labels, and call `plt.show()`.
6. **Q4 Multi-line:** Plot highest and lowest temperatures across 5 days on the same axis with distinct colors and labels, display legend using `plt.legend()`, and call `plt.show()`.
7. **Q5 Subplots:** Create a figure with two subplots side-by-side using `plt.subplot(1, 2, 1)` (Highs) and `plt.subplot(1, 2, 2)` (Lows), add overall title `plt.suptitle()`, and call `plt.show()`.
8. **Q6 Bar Charts:** Plot programming language popularity as vertical bar chart using `plt.bar()`, horizontal bar chart using `plt.barh()`, and multi-colored bar chart by passing a list of colors.
9. **Q7 Grouped Bars:** Plot men and women scores across groups by offsetting bar positions using `x - width` and `x + width`.
10. **Q8 Pie Chart:** Create pie chart of language popularity with percentage labels using `plt.pie(..., autopct="%1.1f%%")`.
11. **Q9 CSV Pie Chart:** Read `medal.csv` using `pd.read_csv()`, extract Country and Gold_Medal arrays, plot Olympic pie chart, and call `plt.show()`.
12. **Q10 Scatter Plot:** Plot Mathematics and Science marks of 10 students using `plt.scatter()` with distinct colors and a legend.
13. **STOP**

### Code Explained in Logical Blocks

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Block 1: Dotted Line and Labeled Plots (Q1 & Q2)
x = np.array([1, 2, 6, 18]); y = np.array([3, 10, 12, 20])
plt.plot(x, y, color="red", linestyle=":", marker=".", markersize=10,
         markerfacecolor="green", markeredgecolor="green")
plt.show()

# Block 2: File-Based Plot & Legends (Q3 & Q4)
file = open("exercise3.3.txt")
x = np.array(list(map(int, file.readline().strip().split(","))))
y = np.array(list(map(int, file.readline().strip().split(","))))
file.close()
plt.title("Temperature Information"); plt.plot(x, y, marker="*"); plt.show()

# Block 3: Subplots (Q5)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.plot([1,2,3,4,5], [30,32,31,32,33], color="red"); plt.title("Highs")
plt.subplot(1, 2, 2); plt.plot([1,2,3,4,5], [27,26,29,28,29], color="blue"); plt.title("Lows")
plt.suptitle("Temperature Difference Over 5 Days"); plt.show()

# Block 4: Bar and Pie Charts (Q6, Q8, Q9)
languages = ["Java", "Python", "PHP", "JavaScript", "C#", "C++"]
popularity = [22.2, 17.6, 8.8, 8, 7.7, 6.7]
plt.bar(languages, popularity); plt.show()
plt.pie([30, 40, 10, 10, 5, 5], labels=languages, autopct="%1.1f%%"); plt.show()

df = pd.read_csv("medal.csv")
plt.pie(df["Gold_Medal"].to_numpy(), labels=df["Country"].to_numpy(), autopct="%1.1f%%")
plt.title("Gold Medals - 2016 Summer Olympics"); plt.show()
```

### Important Syntax to Memorize

- `plt.subplot(1, 2, 1)` — 1 row, 2 columns, first plot active.
- `plt.barh(categories, values)` — Horizontal bar plot.
- `plt.pie(medals, labels=countries, autopct="%1.1f%%")` — Pie chart with 1 decimal percentage.
- `file.readline().strip().split(",")` — Reads comma-delimited line from text file.

### Expected Output

Each block displays an independent GUI figure window. Total of 10 windows shown sequentially upon closing previous windows.

> **Memory Trick: "Line -> File -> Legend -> Subplot -> Bar -> Pie -> Scatter"**

### Common Lab-Exam Mistakes

- Forgetting `plt.show()` at the end of each question (nothing will display).
- Running from outside the directory containing `medal.csv` or `exercise3.3.txt`, causing a `FileNotFoundError`.

**Viva Q1: What does the autopct parameter do in plt.pie()?**
**Answer:** `autopct` formats and displays the percentage share inside each pie slice (e.g., `"%1.1f%%"` formats floats to 1 decimal place followed by a % sign).

**Viva Q2: What is the purpose of plt.subplot(nrows, ncols, index)?**
**Answer:** It divides the plotting area into a grid of `nrows` by `ncols` and activates the subplot at position `index` (1-indexed).

## Program 04: `e04_pandas.py`

**Program 4** **Aim:** To write programs using Pandas demonstrating Series conversions, date ranges, DataFrame creations, multi-column sorting, indexing, grouping, filling missing values, and merging.

**GENERAL ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `pandas` as `pd` and `numpy` as `np`.
3. Convert a Python list `[1, 4, 6, 3]` into a Pandas Series using `pd.Series()` and print.
4. Generate a series of calendar dates from '2021-05-01' to '2021-05-12' using `pd.date_range()` and print.
5. Convert a dictionary of lists (Name, Age, City) into a DataFrame using `pd.DataFrame()` and display.
6. Convert a 2D list into a DataFrame specifying column names `["ID", "Name", "Score"]`.
7. Read `medal.csv` into a DataFrame using `pd.read_csv("medal.csv")` and display.
8. Sort a DataFrame by multiple columns ('Score' descending, 'Name' ascending) using `df.sort_values()`.
9. Reset custom index labels ('p1', 'p2', 'p3') to default numerical indexing using `df.reset_index(drop=True)`.
10. Select and display the first two rows of a DataFrame using indexer `df.iloc[0:2, :]`.
11. Compute average salary per occupation from an employee DataFrame using `df.groupby("Job")["Salary"].mean()`.
12. Create a DataFrame containing `np.nan` values and impute all missing values with 0 using `df.fillna(0)`.
13. Convert numeric profit values into booleans by applying condition `df["profit"] > 0`.
14. Merge two separate employee DataFrames on 'ID' using inner join: `pd.merge(df1, df2, how="inner", on="ID")`.
15. **STOP**

### Code Explained in Logical Blocks

```
import pandas as pd
import numpy as np

# Block 1: Series, Date Ranges, and Creation (Q1 - Q4)
series = pd.Series([1, 4, 6, 3])
dates = pd.date_range("2021-05-01", "2021-05-12")
df_dict = pd.DataFrame({"Name": ["Alice", "Bob", "Charlie"], "Age": [24, 30, 28],
                        "City": ["Delhi", "Mumbai", "Bangalore"]})
df_list = pd.DataFrame([[1, "Alice", 10], [2, "Bob", 15], [3, "Don", 25]],
                       columns=["ID", "Name", "Score"])

# Block 2: CSV Reading, Multi-sort, and Indexing (Q5 - Q8)
df_medal = pd.read_csv("medal.csv")
sorted_df = df_list.sort_values(by=["Score", "Name"], ascending=[False, True])
df_reset = df_dict.copy()
df_reset.index = ["p1", "p2", "p3"]
df_reset = df_reset.reset_index(drop=True)
first_two = df_dict.iloc[0:2, :]

# Block 3: Groupby, Fillna, Booleans, and Merge (Q9 - Q12)
df_emp = pd.DataFrame([["Alice", "Teacher", 25000], ["Bob", "Doctor", 55000],
                       ["Charlie", "Teacher", 20000], ["Dylan", "Doctor", 35000],
                       ["Emma", "Engineer", 65000]], columns=["Name", "Job", "Salary"])
avg_by_job = df_emp.groupby("Job")["Salary"].mean()

df_nan = pd.DataFrame.from_dict({"A": [1, 2, np.nan, 4], "B": [np.nan, 2, 3, 4]}, orient="index")
df_filled = df_nan.fillna(0)

df_profit = pd.DataFrame({"cname": ["A", "B", "C"], "profit": [1000, -500, 0]})
df_profit["profit"] = df_profit["profit"] > 0

df1 = pd.DataFrame([[101, "John", 30000], [102, "Amith", 35000], [103, "Sona", 20000]],
                   columns=["ID", "Name", "Stipend"])
df2 = pd.DataFrame([[101, "Developer"], [102, "Developer"], [103, "Tester"]],
                   columns=["ID", "Position"])
merged_df = pd.merge(df1, df2, how="inner", on="ID")
```

### Important Syntax to Memorize

- `df.sort_values(by=["Score", "Name"], ascending=[False, True])`
- `df.iloc[0:2, :]` — Row and column numerical slicing.
- `df.groupby("Job")["Salary"].mean()` — Category aggregation.
- `df.fillna(0)` — Missing value imputation.
- `pd.merge(df1, df2, how="inner", on="ID")` — Relational join on key column.

### Expected Output (Snippets)

```
# Q9 Groupby Output:
Job
Doctor      45000.0
Engineer    65000.0
Teacher     22500.0
Name: Salary, dtype: float64

# Q12 Merge Output:
    ID   Name  Stipend   Position
0  101   John    30000  Developer
1  102  Amith    35000  Developer
2  103   Sona    20000     Tester
```

**Memory Trick: "Series -> Dates -> DataFrame -> Sort -> Slice -> Group -> Fillna -> Merge"**

### Common Lab-Exam Mistakes

- Forgetting `drop=True` in `reset_index(drop=True)` which inserts an unwanted 'index' column.
- Confusing `iloc` (integer position) with `loc` (label-based).

> **Viva Q1: What is the difference between loc and iloc in Pandas?**
> **Answer:** `loc` selects rows and columns using text/index labels, while `iloc` selects data strictly by 0-based integer index positions.
>
> **Viva Q2: What happens if drop=True is omitted in df.reset_index()?**
> **Answer:** The old index is added as a new column in the DataFrame instead of being discarded.

## Program 05: `e05_knn.py`

**Program 5** **Aim:** To write a program implementing the K-Nearest Neighbors (KNN) algorithm using the Iris dataset, evaluate its classification accuracy, predict an unseen sample, and plot predicted vs actual classes.

**GENERAL ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `load_iris` from `sklearn.datasets`, `train_test_split` from `sklearn.model_selection`, `KNeighborsClassifier` from `sklearn.neighbors`, `metrics` from `sklearn`, and `matplotlib.pyplot` as `plt`.
3. Load the Iris dataset using `load_iris()` and assign feature values to $x$ (`iris.data`) and target class labels to $y$ (`iris.target`).
4. Split features and labels into training (75%) and testing (25%) sets using `train_test_split(x, y, test_size=0.25)`.
5. Initialize the KNN classifier specifying 5 nearest neighbors: `model = KNeighborsClassifier(n_neighbors=5)`.
6. Fit the model using the training data: `model.fit(x_train, y_train)`.
7. Predict classes for the test features: `y_prediction = model.predict(x_test)`.
8. Compute and display the classification accuracy score: `metrics.accuracy_score(y_test, y_prediction)`.
9. Define a new sample `sample = [[2, 2, 2, 2]]`, predict its class using `model.predict(sample)[0]`, and print the target flower name from `iris.target_names`.
10. Create subplot 1 (Predicted classification): scatter plot test feature column 0 vs 1 colored by `y_prediction`, labeled "Sepal length" and "Sepal width".
11. Create subplot 2 (Actual classification): scatter plot test feature column 0 vs 1 colored by `y_test`, labeled "Sepal length" and "Sepal width".
12. Display the side-by-side plots using `plt.show()`.
13. **STOP**

### Code Explained in Logical Blocks

```
# Block 1: Imports and Dataset Loading
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
import matplotlib.pyplot as plt

iris = load_iris()
x = iris.data
y = iris.target

# Block 2: Data Splitting, Model Training, and Evaluation
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25)
model = KNeighborsClassifier(n_neighbors=5)
model.fit(x_train, y_train)

y_prediction = model.predict(x_test)
print("Accuracy = ", metrics.accuracy_score(y_test, y_prediction))

# Block 3: Prediction on a New Sample
sample = [[2, 2, 2, 2]]
pred = model.predict(sample)[0]
print("New Sample belongs to ", iris.target_names[pred])

# Block 4: Side-by-Side Visualization Subplots
plt.subplot(1, 2, 1)
plt.scatter(x_test[:, 0], x_test[:, 1], c=y_prediction)
plt.title("Predicted classification")
plt.xlabel("Sepal length")
plt.ylabel("Sepal width")

plt.subplot(1, 2, 2)
plt.scatter(x_test[:, 0], x_test[:, 1], c=y_test)
plt.title("Actual classification")
plt.xlabel("Sepal length")
plt.ylabel("Sepal width")

plt.show()
```

### Important Syntax to Memorize

- `model = KNeighborsClassifier(n_neighbors=5)` — Instantiates KNN with $k=5$.
- `sample = [[2, 2, 2, 2]]` — Sample must be a 2D array/nested list of shape $(1, 4)$.
- `iris.target_names[pred]` — Converts numeric prediction ($0, 1, 2$) to species string.
- `plt.scatter(x_test[:, 0], x_test[:, 1], c=y_prediction)` — Colors points by predicted class.

### Expected Output

```
Accuracy =  0.9736842105263158  (Varies slightly per run between 0.92 and 1.0)
New Sample belongs to  setosa
(Displays side-by-side plot comparing predicted and actual clusters)
```

**Memory Trick: "Iris -> Split -> KNN(5) -> Fit -> Predict -> Accuracy -> Side-by-Side Plots"**

### Common Lab-Exam Mistakes

- Passing `[2, 2, 2, 2]` (1D list) to `model.predict()` instead of `[[2, 2, 2, 2]]` (2D list).
- Spelling the parameter as `n_neighbor` without the `s` (raises `TypeError`).

**Viva Q1: How does the KNN algorithm classify an unknown query point?**

**Answer:** KNN calculates the Euclidean distance between the query point and all training points, identifies the $k$ nearest data points, and assigns the majority class among those $k$ neighbors.

**Viva Q2: Why is k usually chosen as an odd number in binary classification?**

**Answer:** An odd value of $k$ prevents tie votes between two competing classes.

## Program 06: `e06_naive_bayes.py`

**Program 6** **Aim:** To write a program implementing the Naive Bayes algorithm using the Iris dataset and evaluate its accuracy and full classification report.

**GENERAL ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `train_test_split` from `sklearn.model_selection`, `GaussianNB` from `sklearn.naive_bayes`, `metrics` from `sklearn`, and `load_iris` from `sklearn.datasets`.
3. Load the Iris dataset and assign feature measurements to $x$ and species targets to $y$.
4. Split the data into 70% training and 30% testing subsets with fixed seed: `train_test_split(x, y, test_size=0.3, random_state=40)`.
5. Instantiate the Gaussian Naive Bayes classifier: `gnb = GaussianNB()`.
6. Fit the classifier using the training features and targets: `gnb.fit(x_train, y_train)`.
7. Predict target labels for the test dataset: `y_pred = gnb.predict(x_test)`.
8. Calculate classification accuracy using `metrics.accuracy_score(y_test, y_pred)`.
9. Generate a comprehensive classification report including precision, recall, and f1-score using `metrics.classification_report(y_test, y_pred, target_names=iris.target_names)`.
10. Print the accuracy score and classification report to the console.
11. **STOP**

### Code Explained in Logical Blocks

```
# Block 1: Imports and Dataset Loading
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics
from sklearn.datasets import load_iris

iris = load_iris()
x = iris.data
y = iris.target

# Block 2: Data Splitting with Seed
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=40
)

# Block 3: Model Initialization, Training, and Prediction
gnb = GaussianNB()
gnb.fit(x_train, y_train)
y_pred = gnb.predict(x_test)

# Block 4: Evaluation Metrics
acc = metrics.accuracy_score(y_test, y_pred)
report = metrics.classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
)

print(acc)
print(report)
```

### Important Syntax to Memorize

- `from sklearn.naive_bayes import GaussianNB` — Exact import path.
- `gnb = GaussianNB()` — Note capital `G` and capital `NB`.
- `metrics.classification_report(..., target_names=iris.target_names)` — Generates tabular summary.

### Expected Output (Deterministic because random_state=40)

```
1.0
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        14
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        18

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

> Memory Trick: "Load -> Split(40) -> GaussianNB -> Fit -> Predict -> Report"

### Common Lab-Exam Mistakes

- Capitalization errors: Writing `gaussianNB` or `GaussianNb`.
- Passing `y_train` instead of `y_test` to `accuracy_score()`.

**Viva Q1: Why is the Naive Bayes algorithm called "naive"?**

**Answer:** It is called naive because it makes the strong, unrealistic assumption that all input features are completely independent of each other given the class label.

**Viva Q2: When is GaussianNB specifically used?**

**Answer:** GaussianNB is used when the input features are continuous numerical values assumed to follow a normal (Gaussian) distribution.

## Program 07: `e07_decision_tree.py`

**Program 7** **Aim:** To write a program implementing the Decision Tree Classifier on the Iris dataset, evaluate its performance, and visualize the generated decision tree.

**GENERAL ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `DecisionTreeClassifier` from `sklearn.tree`, `load_iris` from `sklearn.datasets`, `train_test_split` from `sklearn.model_selection`, `metrics` and `tree` from `sklearn`, and `matplotlib.pyplot` as `plt`.
3. Load the Iris dataset and assign features to $x$ and target classes to $y$.
4. Partition the data into training (70%) and testing (30%) sets using `train_test_split(x, y, test_size=0.3)`.
5. Instantiate the Decision Tree classifier with hyperparameters: `clf = DecisionTreeClassifier(criterion="entropy", max_depth=4, min_samples_leaf=2)`.
6. Fit the classifier using the training features and targets: `clf.fit(x_train, y_train)`.
7. Predict class labels on the test dataset: `y_pred = clf.predict(x_test)`.
8. Generate and print the classification report: `metrics.classification_report(y_test, y_pred, target_names=iris.target_names)`.
9. Create a figure canvas of size $25 imes 20$ inches: `plt.figure(figsize=(25, 20))`.
10. Plot the visual tree structure using `tree.plot_tree()`, supplying feature names, class names, and `filled=True`.
11. Display the graphical tree diagram using `plt.show()`.
12. **STOP**

### Code Explained in Logical Blocks

```
# Block 1: Imports and Dataset
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn import tree
import matplotlib.pyplot as plt

iris = load_iris()
x = iris.data
y = iris.target

# Block 2: Splitting and Entropy Tree Training
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3)
clf = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=4,
    min_samples_leaf=2
)
clf = clf.fit(x_train, y_train)

# Block 3: Prediction and Classification Report
y_pred = clf.predict(x_test)
report = metrics.classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
)
print(report)

# Block 4: Visualizing the Tree
fig = plt.figure(figsize=(25, 20))
_ = tree.plot_tree(
    clf,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)
plt.show()
```

### Important Syntax to Memorize

- `DecisionTreeClassifier(criterion="entropy", max_depth=4, min_samples_leaf=2)`
- `tree.plot_tree(clf, feature_names=..., class_names=..., filled=True)`
- `filled=True` — Colors nodes according to the dominant class.

### Expected Output

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.93      0.93      0.93        15
   virginica       0.93      0.93      0.93        15

    accuracy                           0.96        45
   macro avg       0.96      0.96      0.96        45
weighted avg       0.96      0.96      0.96        45
(Renders a large full-screen visualization of the binary decision tree)
```

**Memory Trick: "Entropy Tree -> Fit -> Report -> Plot_Tree(filled=True)"**

### Common Lab-Exam Mistakes

- Omitting `filled=True` resulting in plain black-and-white text boxes without color.
- Misspelling `criterion="entropy"`.

**Viva Q1: What is entropy in a Decision Tree?**

**Answer:** Entropy measures the impurity or disorder of a dataset. A pure node has an entropy of 0.

**Viva Q2: What is the purpose of max_depth and min_samples_leaf?**

**Answer:** They are pruning hyperparameters that control tree growth to prevent overfitting on training data.

## Program 08: `e08_naive_bayes.py`

**Program 8** **Aim:** To write a program implementing Naive Bayes classification using the Iris dataset, compute accuracy percentage, and classify live measurements input by the user from the console.

**GENERAL ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `datasets`, `naive_bayes`, `train_test_split`, and `accuracy_score` from `sklearn`.
3. Load the Iris dataset using `datasets.load_iris()` and separate features $x$ and targets $y$.
4. Instantiate the Gaussian Naive Bayes classifier: `classifier = naive_bayes.GaussianNB()`.
5. Split data into 80% training and 20% testing sets using `train_test_split(x, y, train_size=0.8)`.
6. Fit the classifier using the training features and targets: `classifier.fit(x_train, y_train)`.
7. Predict test set labels: `result = classifier.predict(x_test)`.
8. Compute and print percentage accuracy: `accuracy_score(y_test, result) * 100`.
9. Prompt the user to enter feature measurements corresponding to `iris.feature_names`.
10. Parse user input by splitting on whitespace and converting each value to `float` inside a 2D nested list: `features = [[float(i) for i in input(...).split()]]`.
11. Predict the class of the user input: `result = classifier.predict(features)`.
12. Map predicted numeric index to target flower name using `iris["target_names"][result]` and display.
13. **STOP**

### Code Explained in Logical Blocks

```
# Block 1: Imports and Dataset
from sklearn import datasets
from sklearn import naive_bayes
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = datasets.load_iris()
x, y = iris.data, iris.target

# Block 2: Model Training and Accuracy Percentage
classifier = naive_bayes.GaussianNB()
x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.8)
classifier.fit(x_train, y_train)

result = classifier.predict(x_test)
accuracy = accuracy_score(y_test, result) * 100
print(f"Accuracy is {accuracy}")

# Block 3: Dynamic User Input and Prediction
features = [
    [float(i) for i in input(
        f"Enter {iris.feature_names}"
    ).split()]
]

result = classifier.predict(features)
print(iris["target_names"][result])
```

### Important Syntax to Memorize

- `train_test_split(x, y, train_size=0.8)` — Uses `train_size` instead of `test_size`.
- `accuracy_score(y_test, result) * 100` — Multiplies by 100 for percentage.
- `features = [[float(i) for i in input(...).split()]]` — Parses space-separated decimals into a $(1, 4)$ 2D list.
- `iris["target_names"][result]` — Array indexing with prediction array.

### Expected Output

```
Accuracy is 96.66666666666667
Enter ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']: 5.1 3.5 1.4 0.2
['setosa']
```

**Memory Trick: "Split 80/20 -> Accuracy% -> Input(float) -> Predict User Sample"**

### Common Lab-Exam Mistakes

- Using `int(i)` instead of `float(i)`. Entering real Iris measurements like `5.1 3.5 1.4 0.2` will throw a fatal `ValueError: invalid literal for int()`.
- Entering commas instead of spaces when typing the four numbers into the terminal.

**Viva Q1: Why must user inputs be converted into a nested list [[...]] before calling predict()?**

**Answer:** Scikit-Learn estimators expect a 2D array-like structure of shape `(n_samples, n_features)`. Double brackets create a 2D structure with 1 sample.

**Viva Q2: What mathematical theorem underpins Naive Bayes?**

**Answer:** Bayes' Theorem: $P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$.

## Program 09: `e09_kmeans.py`

**Program 9** **Aim:** To write a program implementing K-Means clustering using synthetically generated coordinate data, locate centroids, plot clusters, and predict clusters for new data points.

**GENERAL ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `numpy` as `np`, `KMeans` from `sklearn.cluster`, `make_blobs` from `sklearn.datasets`, and `matplotlib.pyplot` as `plt`.
3. Generate 300 synthetic 2D data points using `data, _ = make_blobs(n_samples=300)`.
4. Initialize the K-Means clustering model specifying 5 clusters: `kmeans = KMeans(n_clusters=5, random_state=42, n_init="auto")`.
5. Fit the model to the coordinate data using `kmeans.fit(data)`.
6. Extract the cluster labels assigned to each data point via `kmeans.labels_`.
7. Extract the cluster center spatial coordinates via `kmeans.cluster_centers_`.
8. Create a scatter plot of the data points mapping cluster labels to colors: `plt.scatter(data[:, 0], data[:, 1], c=labels, cmap="viridis")`.
9. Overlay the centroids as red 'x' markers: `plt.scatter(centroids[:, 0], centroids[:, 1], color="red", marker="x")`.
10. Plot new test points in red: `plt.scatter([2, 6], [-4, 6], color="red")`.
11. Display the cluster figure using `plt.show()`.
12. Predict and print the cluster assignments for coordinate points `[[2, -4], [6, 6], [6, -2]]` using `kmeans.predict()`.
13. **STOP**

### Code Explained in Logical Blocks

```
# Block 1: Imports and Synthetic Data Generation
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

data, _ = make_blobs(n_samples=300)

# Block 2: K-Means Initialization and Fitting
kmeans = KMeans(
    n_clusters=5,
    random_state=42,
    n_init="auto"
).fit(data)

labels = kmeans.labels_
centroids = kmeans.cluster_centers_

# Block 3: Cluster Visualization
plt.scatter(
    data[:, 0],
    data[:, 1],
    c=labels,
    cmap="viridis"
)
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    color="red",
    marker="x"
)
plt.legend(list(set(labels)))
plt.scatter([2, 6], [-4, 6], color="red")
plt.show()

# Block 4: Predicting Clusters for Unseen Points
print(kmeans.predict([[2, -4], [6, 6], [6, -2]]))
```

### Important Syntax to Memorize

- `data, _ = make_blobs(n_samples=300)` — Generates synthetic cluster data.
- `kmeans.labels_` — Trailing underscore attribute containing point labels.
- `kmeans.cluster_centers_` — Trailing underscore attribute with centroid $(x, y)$ coordinates.
- `kmeans.predict([[2, -4], [6, 6], [6, -2]])` — Assigns new points to closest centroid.

### Expected Output

```
[3 1 1]   (Array of 3 cluster indices between 0 and 4 depending on synthetic blob positions)
(Displays scatter plot with 5 colored clusters, red 'x' centroids, and test points)
```

**Memory Trick: "Blobs -> KMeans(5) -> Fit -> Labels & Centroids -> Scatter Viridis -> Predict"**

### Common Lab-Exam Mistakes

- Attempting a `train_test_split`. K-Means is **unsupervised** and does not have target labels $y$.
- Forgetting the trailing underscores in `kmeans.labels_` and `kmeans.cluster_centers_`.

**Viva Q1: Is K-Means supervised or unsupervised, and why?**

**Answer:** It is unsupervised because it clusters data points entirely based on feature similarities (Euclidean distances) without receiving ground truth target labels.

**Viva Q2: What does the centroid represent in K-Means?**

**Answer:** The centroid is the arithmetic mean (center of mass) of all the data points currently assigned to that cluster.

## Program 10: `e10_multiple_linear_regression.py`

**Program 10** **Aim:** To write a program implementing Multiple Linear Regression using the Diabetes dataset, print coefficients and intercept, evaluate using MSE and $R^2$ score, and plot regression prediction curves.

**GENERAL ALGORITHM (Exam Writing Format):**

1. **START**
2. Import `numpy` as `np`, `matplotlib.pyplot` as `plt`, `datasets` and `linear_model` from `sklearn`, and `mean_squared_error` and `r2_score` from `sklearn.metrics`.
3. Load the continuous Diabetes dataset using `datasets.load_diabetes(return_X_y=True)` to unpack features $X$ and progression target $y$.
4. Select two predictor features: column 0 (Age) and column 2 (BMI) using NumPy indexing: `diabetes_X[:, [0, 2]]`.
5. Split data into training (all rows except last 20) and testing (last 20 rows) subsets: `[:-20]` and `[-20:]`.
6. Instantiate the linear regression model: `regr = linear_model.LinearRegression()`.
7. Fit the model using training sets: `regr.fit(diabetes_X_train, diabetes_y_train)`.
8. Generate predictions for the test set: `diabetes_y_pred = regr.predict(diabetes_X_test)`.
9. Print model slopes via `regr.coef_` and y-intercept via `regr.intercept_`.
10. Evaluate and print Mean Squared Error (MSE) and $R^2$ score.
11. Create Figure 1 with two subplots comparing Feature 0 (Age) vs Target and Feature 2 (BMI) vs Target with actual and predicted markers, and call `plt.show()`.
12. Create Figure 2 plotting Actual vs Predicted progression with an ideal fit diagonal line ($y=x$), and call `plt.show()`.
13. **STOP**

### Code Explained in Logical Blocks

```
# Block 1: Imports, Data Loading, and Slicing Features
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score

diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)
diabetes_X = diabetes_X[:, [0, 2]]

# Block 2: 20-Sample Slicing Split and Model Fitting
diabetes_X_train, diabetes_X_test = diabetes_X[:-20], diabetes_X[-20:]
diabetes_y_train, diabetes_y_test = diabetes_y[:-20], diabetes_y[-20:]

regr = linear_model.LinearRegression()
regr.fit(diabetes_X_train, diabetes_y_train)
diabetes_y_pred = regr.predict(diabetes_X_test)

# Block 3: Model Equations and Metrics
print("Coefficients:", regr.coef_)
print("Intercept:", regr.intercept_)
print("Mean squared error: %.2f" % mean_squared_error(diabetes_y_test, diabetes_y_pred))
print("R^2 score: %.2f" % r2_score(diabetes_y_test, diabetes_y_pred))

# Block 4: Subplots for Individual Features
plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
plt.scatter(diabetes_X_test[:, 0], diabetes_y_test, color='red', label='Actual')
plt.scatter(diabetes_X_test[:, 0], diabetes_y_pred, color='blue', marker='^', label='Predicted')
plt.xlabel('Feature 0 (Age)'); plt.ylabel('Diabetes Progression'); plt.legend()

plt.subplot(1, 2, 2)
plt.scatter(diabetes_X_test[:, 1], diabetes_y_test, color='red', label='Actual')
plt.scatter(diabetes_X_test[:, 1], diabetes_y_pred, color='green', marker='D', label='Predicted')
plt.xlabel('Feature 2 (BMI)'); plt.ylabel('Diabetes Progression'); plt.legend()
plt.tight_layout(); plt.show()

# Block 5: Actual vs Predicted Ideal Fit Plot
plt.figure(figsize=(8, 6))
plt.scatter(diabetes_y_test, diabetes_y_pred, color='purple', marker='^', label='Predicted vs Actual')
plt.plot([diabetes_y_test.min(), diabetes_y_test.max()], [diabetes_y_test.min(), diabetes_y_test.max()],
         'r--', label='Ideal Fit')
plt.xlabel("Actual"); plt.ylabel("Predicted"); plt.legend(); plt.show()
```

### Important Syntax to Memorize

- `datasets.load_diabetes(return_X_y=True)` — Unpacks directly into $X$ and $y$.
- `diabetes_X[:, [0, 2]]` — Slices all rows for columns 0 (Age) and 2 (BMI).
- `diabetes_X[:-20], diabetes_X[-20:]` — Reserving last 20 rows for test set.
- `regr.coef_` (weights/slopes) & `regr.intercept_` (bias).

### Expected Output

```
Coefficients: [ 10.96347101 928.30799309]
Intercept: 152.92484742468243
Mean squared error: 2548.07
R^2 score: 0.47
(Opens Figure 1 with 2 subplots, followed by Figure 2 with ideal fit line)
```

**Memory Trick: "Diabetes [0, 2] -> Slice 20 -> LinearRegression -> Coef/Intercept -> MSE & R2 -> Plots"**

### Common Lab-Exam Mistakes

- Attempting to print `accuracy_score()`. Regression predicts continuous real numbers; accuracy is strictly for classification.
- Forgetting `return_X_y=True` which causes `load_diabetes()` to return a Bunch dictionary instead of two arrays.

**Viva Q1: What does the R2 score represent in linear regression?**

**Answer:** The $R^2$ score (coefficient of determination) represents the proportion of variance in the dependent variable that is predictable from the independent features. 1.0 represents perfect prediction.

**Viva Q2: What do regr.coef_ and regr.intercept_ represent?**

**Answer:** In the equation $y = m_1 x_1 + m_2 x_2 + c$, `regr.coef_` represents the slopes/weights ($m_1, m_2$) and `regr.intercept_` represents the constant baseline bias ($c$).

## Section 5: Comparative Analysis & Rapid Revision

### 1. KNN vs. Naive Bayes vs. Decision Tree

| Algorithm | Primary Concept | Core Assumption | Model Statement | Hyperparameters in Repo |
|---|---|---|---|---|
| **KNN** ( `e05` ) | Instance-based / Lazy learner | Nearby points belong to same class | `KNeighborsClassifier(n_neighbors=5)` | `n_neighbors=5` |
| **Naive Bayes** ( `e06, e08` ) | Probabilistic / Bayes' Rule | Features are conditionally independent | `GaussianNB()` | None required |
| **Decision Tree** ( `e07` ) | Information Gain / Entropy splits | Data can be partitioned by axis-aligned rules | `DecisionTreeClassifier(...)` | `criterion="entropy", max_depth=4, min_samples_leaf=2` |

### 2. Supervised vs. Unsupervised Learning

| Feature | Supervised Learning (e05, e06, e07, e08, e10) | Unsupervised Learning (e09) |
|---|---|---|
| **Target Labels ($y$)** | Mandatory (Training provides inputs $X$ and true outputs $y$) | **None** (Algorithm receives only coordinates $X$) |
| **Data Partitioning** | Requires training and testing partition ( `train_test_split` ) | No split; fits directly to all available data points |
| **Fitting Call** | `model.fit(x_train, y_train)` | `model.fit(data)` |
| **Goal** | Map future inputs to known target labels or numbers | Discover underlying spatial patterns, clusters, and centroids |

### 3. Classification vs. Regression

| Aspect | Classification (e05, e06, e07, e08) | Regression (e10) |
|---|---|---|
| **Output Type** | Discrete categorical labels (e.g. Setosa, Versicolor, Virginica) | Continuous floating-point quantities (e.g. Disease score 152.9) |
| **Dataset Used** | Iris Dataset ( `load_iris` ) | Diabetes Dataset ( `load_diabetes` ) |
| **Primary Metrics** | Accuracy Score, Precision, Recall, F1-Score | Mean Squared Error (MSE), Coefficient of Determination ($R^2$) |

### 4. Pre-Exam Mental Checklist

- [ ] **Correct Imports:** Did I import the exact model class (e.g. `GaussianNB` with capital G and NB)?
- [ ] **Variable Unpacking:** Did I unpack `train_test_split()` in the exact order: `x_train, x_test, y_train, y_test` ?
- [ ] **Fit vs. Predict:** Did I fit with training data ( `x_train, y_train` ) and predict with test data ( `x_test` )?
- [ ] **Single Sample Dimension:** If predicting one sample, did I wrap it in double brackets `[[...]]` to form a 2D array?
- [ ] **Input Type:** In `e08` , did I parse user inputs with `float(i)` so decimals do not crash?
- [ ] **Plotting Calls:** Did I end every Matplotlib script with `plt.show()` ?
- [ ] **Working Directory:** Am I running the Python commands inside the directory containing `medal.csv` and `exercise3.3.txt` ?

## Section 6: Last-Minute Rapid Revision Sheet

Memorize these high-frequency code templates right before entering the exam hall:

### 1. The Universal Scikit-Learn Template (Iris Classification)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn import metrics
# Model choice (pick one):
from sklearn.neighbors import KNeighborsClassifier; model = KNeighborsClassifier(n_neighbors=5)
from sklearn.naive_bayes import GaussianNB; model = GaussianNB()
from sklearn.tree import DecisionTreeClassifier; model = DecisionTreeClassifier(criterion="entropy")

iris = load_iris()
x, y = iris.data, iris.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3)
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

### 2. The K-Means Template

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

data, _ = make_blobs(n_samples=300)
kmeans = KMeans(n_clusters=5, random_state=42, n_init="auto").fit(data)
plt.scatter(data[:, 0], data[:, 1], c=kmeans.labels_, cmap="viridis")
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], color="red", marker="x")
plt.show()
print(kmeans.predict([[2, -4], [6, 6], [6, -2]]))
```

### 3. The Multiple Linear Regression Template

```
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score

X, y = datasets.load_diabetes(return_X_y=True)
X = X[:, [0, 2]]
X_train, X_test, y_train, y_test = X[:-20], X[-20:], y[:-20], y[-20:]
regr = linear_model.LinearRegression().fit(X_train, y_train)
pred = regr.predict(X_test)
print("Coefficients:", regr.coef_, "Intercept:", regr.intercept_)
print("MSE: %.2f" % mean_squared_error(y_test, pred))
print("R2: %.2f" % r2_score(y_test, pred))
```

### 4. Essential One-Liners

- `arr[arr > 4]` — Boolean filter numbers greater than 4.
- `np.where(arr == num)` — Find indices where element matches.
- `np.dot(m1, m2)`, `np.transpose(m1)`, `np.linalg.det(m1)`, `np.linalg.inv(m1)`.
- `df.sort_values(by=["Score", "Name"], ascending=[False, True])`.
- `df.groupby("Job")["Salary"].mean()`.
- `pd.merge(df1, df2, how="inner", on="ID")`.
- `plt.subplot(1, 2, 1)` — Subplot 1 of 2.
- `plt.pie(values, labels=categories, autopct="%1.1f%%")`.

---

## Section 7: Review Notes & Observations on Repository Files

This section documents observations and the status of files in your repository:

1. **Approved Fix in `e08_naive_bayes.py` (Line 29):** The input parsing expression was changed from `int(i)` to `float(i)`. This ensures that floating-point Iris measurements (e.g. `5.1 3.5 1.4 0.2`) are parsed without raising a fatal `ValueError`.
2. **Approved Fix in `e05_knn.py` (Lines 44-45 & 57-58):** The scatter plot axis labels were corrected from "Petal length" / "Petal width" to "Sepal length" / "Sepal width" because columns 0 and 1 correspond to Sepal dimensions in Scikit-Learn's Iris dataset.
3. **Compatibility Note on `e09_kmeans.py` (`n_init="auto"`):** In Scikit-Learn versions $\ge 1.2$ (including your installed version 1.9.1), `n_init="auto"` is fully supported. On legacy systems ($< 1.2$), an integer like `n_init=10` is used.
4. **Plotted vs. Predicted Points in `e09_kmeans.py`:** Line 46 plots 2 points `[2, 6]` and `[-4, 6]`, while Line 52 predicts 3 points: `[[2, -4], [6, 6], [6, -2]]`. For an exam answer, you can optionally include the third point in the scatter plot.
5. **Unused Import in `e07_decision_tree.py`:** Line 6 imports `pandas as pd` but does not use it. It is harmless during execution.
6. **Sequential `plt.show()` Calls in `e03_matplotlib.py` and `e10_multiple_linear_regression.py`:** Each `plt.show()` pauses script execution until the user closes the active figure window. In an exam, closing the first window immediately opens the next.